Watermark-Guided Tamper Localization and Recovery: A Key-Seeded Self-Embedding Fragile Watermarking Scheme with Joint Localization and Restoration Assessment

[Author Name(s) — PLACEHOLDER]
Department of Computer Science
Ajeenkya DY Patil School of Engineering, Pune, India
Email: [institutional email]

[Guide Name — PLACEHOLDER]
Department of Computer Science
Ajeenkya DY Patil School of Engineering, Pune, India

Abstract—Consumer editing tools make convincing local image manipulation trivial, yet passive forensic detectors return opaque confidence scores, need annotated training corpora and GPU inference, and cannot restore what was destroyed. We present a deterministic, training-free self-embedding fragile watermarking scheme that both localizes tampering and approximately recovers it from the image itself. Each 8×8 block carries a 32-bit keyed authentication tag over its own six most-significant bit-planes and the recovery descriptor it holds, plus a 96-bit compressed descriptor of a spatially distant partner block chosen by a key-seeded single-cycle permutation, all embedded in two least-significant bit-planes. Verification recomputes each tag; a mismatch localizes tampering, and each flagged block is rebuilt from its partner's descriptor or explicitly marked unrecoverable. On a 32-image corpus, imperceptibility is 43.17 dB / SSIM 0.9824 (DCT descriptor) and 44.23 dB / 0.9844 (mean-pooled). Across 768 main-grid tamper trials spanning four tamper classes and three ratios, block-level recall is 1.0000 and pixel-level precision 0.9489; we deliberately do not headline block-level precision, since the ground-truth rule makes it structurally indistinguishable from 1.0 regardless of detector quality. A 1,802,240-block null condition yields zero false positives, reported as a determinism and implementation-consistency check rather than a statistical bound on the 2^{-32} per-channel false-accept probability. In-region recovery is essentially flat across tamper ratio (28.96 to 28.38 dB) while whole-image PSNR collapses (33.79 to 17.39 dB): measured evidence that mapping coverage, not descriptor quality, dominates recovery, and that one-to-one partner mapping falls well short of reference-sharing schemes at high tamper ratios. Every decision traces to a specific hash mismatch rather than a learned score.

Index Terms—fragile watermarking, self-embedding, tamper localization, image authentication, content recovery, block mapping

I. Introduction

The evidentiary value of a digital image rests on an assumption that is increasingly unsafe: that the pixels presented are the pixels captured. Region-level manipulation – pasting an object from one photograph into another, erasing an object with an inpainting tool, or replacing a cropped region with plausible texture – is now available in free software and, with diffusion-based generative editors, requires neither skill nor time. The consequences are concrete rather than hypothetical. A radiological study with a lesion removed, a vehicle damage photograph submitted in an insurance claim, a surveillance frame entered into a legal record, or a remote-sensing tile used in an environmental dispute all derive their utility from integrity that the image format itself does not protect.

Two broad families of countermeasures exist. Passive (blind) forensics analyses the received image alone, searching for statistical inconsistencies in noise residuals, compression history, colour filter array traces, illumination or resampling artefacts; modern instances of this family are learned, using convolutional or transformer backbones trained on manipulation corpora to output a per-pixel manipulation heat map [12], [13]. Active approaches insert authentication information into the image before distribution, so that verification becomes a decision about self-consistency rather than an inference about provenance. Fragile and semi-fragile watermarking belongs to this second family [3], [11].

Even a perfect detector, however, answers only half of the question a practitioner actually asks. Knowing that a region of a radiograph was altered, and even knowing precisely which pixels were altered, does not restore the diagnostic content that was overwritten. In the passive paradigm, restoration is necessarily a generative act: an inpainting network hallucinates content that is plausible but has no evidentiary relationship to the original. This is the motivating observation of the present work. Detection alone is insufficient, and restoration that invents content is worse than no restoration at all when the artefact is evidence. What is required is a mechanism by which the original content – even a coarse, low-frequency approximation of it – survives the attack and can be reinstated.

Self-embedding watermarking, introduced in the line of work following Fridrich and Goljan [1], [2], provides

exactly this mechanism: a compressed representation of each region of the image is hidden elsewhere within the same image, so that a spatially localized attack destroys the region but not its backup. Our system develops this idea into a complete, reproducible localization-and-recovery pipeline and – critically – evaluates the two capabilities together, which the literature has tended not to do.

A second motivation is deployment economics and accountability. Learned tamper detectors carry three costs that are frequently understated: they require large annotated manipulation datasets whose bias determines what the model can see; they require accelerator hardware at inference time, which excludes embedded acquisition devices, hospital PACS gateways and field equipment; and their outputs are not explainable in a form that survives adversarial cross-examination. A per-pixel softmax score of 0.78 is not an argument. By contrast, the scheme described here is classical deterministic signal processing: every positive detection is the statement "the 32-bit keyed tag stored in block i does not equal the tag recomputed from block i's current most-significant content," which is a verifiable, repeatable, and auditable fact. There is no training set, no random initialization, no GPU, and no run-to-run variance.

The contributions of this paper are as follows.

1) A concrete self-embedding fragile watermarking design that separates authentication payload (stored in-block) from recovery payload (stored in a distant partner block), with a bit budget that fits entirely within two least-significant bit-planes and therefore bounds embedding distortion analytically.
2) A specific construction for avoiding the tamper coincidence problem – the failure mode in which a block and the block holding its own recovery data are destroyed by the same attack. The distant-mapping principle for avoiding this problem was proposed first by Aminuddin and Ernawan's AuSR3 [20]; our contribution is not that idea but the construction: a key-seeded single-cycle permutation over the block index set, built directly in cycle notation so bijectivity and the absence of short cycles hold unconditionally, combined with an enforced minimum-separation repair pass.
3) A joint evaluation protocol over four realistic tamper classes with exact, self-generated ground-truth masks, reporting imperceptibility, localization precision/recall/F1/IoU, recoverability rate, and in-region recovery PSNR/SSIM in a single table, so the localization–recovery trade-off is visible rather than implied.
4) An explicit positioning of the scheme relative to both classical self-embedding watermarking and recent learned proactive watermarking, which now also localizes and recovers tampering [25]–[27]: the axis of difference offered here is determinism, zero training, zero GPU inference, and full explainability – every verification decision reduces to a single recomputed HMAC comparison that any third party holding the key can independently re-run and check bit-for-bit, which a learned localizer's network output cannot offer. This is a claim about auditability, not about outperforming learned methods on accuracy. We further state candidly the scheme's core limitation – fragility to any lossy re-encoding – and the archival, lossless-format deployment contexts in which that limitation is acceptable.
5) An adversarial security analysis of the scheme itself, surfacing and fixing two real defects with verified exploits: an unauthenticated recovery descriptor that left three quarters of every block's embedded payload (96 of 128 bits) with no integrity protection, letting an attacker certify a gutted image as authentic; and a block-mapping seed order that leaked spatial structure well beyond its documented minimum-separation guarantee, letting an attacker bias a coverage-denial attack without ever holding the key. Both are fixed at zero payload cost, and both fixes are verified against the same exploit that found them (Section III, Section VI).

The remainder of the paper is organized as follows. Section II reviews related work. Section III specifies the proposed methodology. Section IV describes the system architecture. Section V gives the experimental setup, Section VI presents and discusses results, and Section VII concludes.

II. Related Work

Robust versus fragile watermarking. Early digital watermarking research was dominated by robustness: the watermark was required to survive compression, filtering and geometric distortion so that ownership could still be asserted. The spread-spectrum formulation of Cox et al. [14], which embeds the mark in perceptually significant transform coefficients, is the canonical reference for this design goal. Authentication inverts the requirement: the watermark should be fragile – it must fail under modification, because failure is the signal. Wong and Memon [10] developed block-wise fragile schemes in which per-block signatures are embedded in the least-significant bits so that verification is both localized and computable without the original image. Survey treatments of the authentication watermarking design space, including the fragile/semi-fragile distinction, are available in the watermarking survey literature [11].

Semi-fragile authentication and the JPEG problem. A purely fragile watermark cannot distinguish a malicious splice from a benign JPEG re-save. Lin and Chang [3] addressed this directly, exploiting an invariant relationship between DCT coefficient pairs that is preserved by quantization but broken by content manipulation. This established the semi-fragile design pattern and remains

the standard point of comparison for schemes claiming compression tolerance; we cite it to delimit our own scope, since the present system is deliberately fully fragile (Section VII discusses the semi-fragile extension path).

Self-embedding and content recovery. The decisive step from detection to recovery is due to Fridrich and Goljan, who proposed embedding a compressed version of each image block within the image itself so a tampered region could be approximately reconstructed from surviving data [1], [2]. Their construction – block-wise compression, key-dependent mapping to a distant carrier block, and LSB embedding – is the direct ancestor of the design in this paper. Lin et al. [4] introduced a hierarchical, multi-level block structure verifying at several granularities, improving localization resolution and robustness of the tamper decision; this hierarchical idea informs the neighbourhood post-processing described in Section III. Zhang and Wang [7] – and, in a separate study that must not be conflated with it, Zhang, Wang and Feng [22] – and Zhang et al. [8] substantially improved reconstruction quality and reframed the payload allocation problem: reference information is shared across blocks so reconstruction solves a linear system, raising the tolerable tamper ratio above the naive one-to-one limit. Korus and Dziech [9] developed content-adaptive self-embedding with a formal rate–distortion analysis of achievable reconstruction quality as a function of tampering rate. More recently, Aminuddin and Ernawan's AuSR series [18]–[20] target the same recovery problem at finer, 2×2 block granularity, and give an explicit name to a failure mode this line of work must contend with: the tamper coincidence problem, in which a block and the block holding its own recovery data are damaged by the same attack. AuSR3 [20] addresses it by mapping each block's recovery data to the most distant available location; the block mapping in Section III implements the same distant-mapping principle as a single-cycle permutation with an enforced minimum-separation repair pass.

Security of localized authentication watermarks. Block-wise independent authentication is vulnerable in an easy-to-overlook way. Holliman and Memon [5] showed that if each block's watermark depends only on that block's content, an adversary holding a set of legitimately watermarked images can assemble a counterfeit image by collaging authentic blocks, and every block will verify. Fridrich [6] analysed the security of localizing fragile watermarks in the same spirit. The practical consequence, adopted in Section III, is that the authentication tag must be bound not only to block content but also to the block's index and to an image-level identifier under a secret key, so a block moved to a different position or transplanted from a different image fails verification.

Learned manipulation localization. Passive, learning-based localization has advanced rapidly, including two-stream RGB/noise-residual architectures for splice localization [12] and self-supervised anomaly-feature networks that localize forgeries without training on the specific manipulation type [13]. These systems require no cooperation from the acquisition device – their principal advantage over any active scheme including ours. Their costs – annotated training data, GPU inference, non-explainable per-pixel scores, sensitivity to distribution shift – are the costs this paper's design avoids. We regard the two families as complementary: an active watermark is available whenever the capture/archival pipeline is under the verifier's control, and passive forensics is the fallback when it is not.

Learned proactive watermarking. A separate and more recent line of work embeds a learned watermark specifically so that a trained network can both localize tampering and recover the original content, closing exactly the gap that has traditionally motivated self-embedding. EditGuard [25] jointly optimizes an invertible watermarking network for tamper localization and copyright protection, reporting over 95% localization precision; DeepMark [26] trains a proactive tamper detector and localizer across both images and video; RecoverMark [27] targets joint immunization and recovery specifically for manipulated face imagery. These systems demonstrate that "detects only, does not recover" is no longer an accurate characterization of active watermarking as a field. The distinction that survives is not capability but auditability: a learned localizer's output is a trained network's inference, not independently recomputable by a third party without the model weights, whereas every decision in the scheme below is a single keyed HMAC comparison that anyone holding the key can recompute and check by hand. We therefore do not claim to detect or recover more accurately than these learned systems; we claim a different, complementary property.

Positioning. Across the self-embedding literature, evaluation is asymmetric: detection-oriented papers report localization accuracy, recovery-oriented papers report reconstruction PSNR under a single attack, and few report both jointly, over several realistic tamper classes, at several tamper-area ratios, against exact ground-truth masks. Supplying that joint characterization for a deliberately minimal, training-free construction is the contribution of this paper; the learned proactive-watermarking family described above increasingly reports localization and recovery together as well, so the joint-characterization contribution claimed here is scoped specifically to the classical, deterministic self-embedding literature, not to that family.

III. Proposed Methodology

A. Notation and Preliminaries

Let the host image be a grayscale array I of size $M \times N$ with 8-bit precision, $I(x, y) \in \{0, \ldots, 255\}$. Colour images are handled by applying the procedure independently per channel, or to the luminance channel of a reversible colour transform when payload budget is

scarce. Spatial dimensions are assumed multiples of the block size $B = 8$; otherwise the image is cropped to the largest block-aligned region and the crop reported, rather than padded – padding would place real blocks' descriptors into phantom blocks that are discarded, making those blocks unrecoverable by construction. Every corpus image used in Section V is already a multiple of both 8 and 4, so no crop occurs in practice.

The image is partitioned into $K = (M/B) \cdot (N/B)$ non-overlapping blocks $\{\mathbf{b}_1, \ldots, \mathbf{b}_K\}$ in raster order, $\mathbf{b}_i \in \{0, \ldots, 255\}^{B \times B}$. Let $c(i) \in \mathbb{R}^2$ denote the pixel-coordinate centroid of block i.

Two bit-planes are reserved for embedding. Define the MSB projection operator, which zeroes the two least-significant bits of every pixel:

$$\mathrm{MSB}(\mathbf{b}) = \lfloor \mathbf{b}/4 \rfloor \cdot 4 \tag{1}$$

Equivalently, $\mathrm{MSB}(\mathbf{b}) = \mathbf{b}\,\mathrm{AND}\,0\mathrm{xFC}$. All payload – both the authentication tag and the recovery descriptor – is computed from $\mathrm{MSB}(\cdot)$ of the source block, never from the raw block, so embedding cannot alter the quantity that was hashed or compressed. The verifier recomputes the identical quantity from the received image without needing the original, and no iterative fixed-point search is required.

The per-block payload capacity is $2 \times B^2 = 128$ bits, allocated as

$$128 = \underbrace{32}_{\text{tag } a_i} + \underbrace{96}_{\text{descriptor } r_j,\ j=m^{-1}(i)} \tag{2}$$

so block i carries its own authentication tag and another block's recovery descriptor. The 32-bit tag yields a per-block false-acceptance probability of $2^{-32} \approx 2.3 \times 10^{-10}$, negligible over the $K = 4096$ blocks of a 512×512 image.

B. Recovery Descriptor Construction

For each block i, a 96-bit descriptor r_i compressing the visual content of $\mathrm{MSB}(\mathbf{b}_i)$ is computed. Two variants are specified and both are evaluated in Section VI.

Variant A (DCT, primary). Let $\mathbf{C}_i = \mathrm{DCT2}(\mathrm{MSB}(\mathbf{b}_i) - 128)$ be the level-shifted 2-D type-II DCT of the block. Let $\sigma(1), \ldots, \sigma(64)$ be the zig-zag scan order; retain the first $L = 12$ coefficients (DC plus eleven lowest-frequency AC terms). Each retained coefficient is quantized to a uniform 8 signed bits,

$$\hat{q}_{i,\ell} = \mathrm{clip}\left(\mathrm{round}\left(\frac{C_i(\sigma(\ell))}{\Delta_\ell}\right), -128, 127\right),\ \ell = 1, \ldots, 12, \tag{3}$$

so $L \times 8 = 12 \times 8 = 96$ bits exactly, with round-half-even rounding. The decreasing-precision intent of natural-image DCT coding is carried entirely by the per-band step size Δ_ℓ rather than by a decreasing bit allocation: Δ_ℓ is taken from the JPEG quality-50 luminance quantization table in zig-zag order, with the DC entry reduced to 8 so that the DC term – which carries the most energy – retains the finest quantization step. An increasing Δ_ℓ is an equivalent, and simpler to implement, way of expressing a decreasing effective bit allocation. Concatenating the 12 signed bytes in scan order gives $r_i \in \{0,1\}^{96}$; two's-complement byte representation makes packing and sign handling a single array view, with no explicit sign bit.

No retained coefficient can clip. With an orthonormal 2-D DCT, Cauchy–Schwarz gives $|C_i(u,v)| \le 128B$ for an 8-bit input block of size B; for $B = 8$ this is 1024. The DC step of 8 gives $1024/8 = 128$, exactly filling the signed 8-bit range; every AC step is at least 10, giving $1024/10 = 102.4 < 127$. No retained coefficient can therefore clip for any 8-bit input, for any $B = 8$ block – the $\mathrm{clip}(\cdot)$ in (3) is a defensive bound only. Reconstruction dequantizes, zero-pads coefficients $\ell > L$, applies the inverse DCT, and re-adds the level shift:

$$\tilde{\mathbf{b}}_i = \mathrm{clip}\big(\mathrm{IDCT2}(\mathcal{Z}(\hat{q}_i \odot \Delta)) + 128,\ 0,\ 255\big) \tag{4}$$

where $\mathcal{Z}(\cdot)$ places values into an 8×8 zero array at the zig-zag positions.

Variant B (mean-pooled, low-complexity). The 8×8 block is reduced to a 4×4 array of 2×2 means,

$$\mu_i(u,v) = \left\lfloor \frac{1}{4} \sum_{p=0}^{1} \sum_{q=0}^{1} \mathrm{MSB}(\mathbf{b}_i)(2u+p, 2v+q) \right\rfloor \tag{5}$$

for $0 \le u, v \le 3$, each quantized to 6 bits by discarding the two least-significant bits, $\hat{\mu}_i = \lfloor \mu_i/4 \rfloor$, giving exactly $16 \times 6 = 96$ bits. Reconstruction upsamples $4\hat{\mu}_i + 2$ to 8×8. Variant B needs no transform – only integer shifts – and targets microcontroller-class deployment; Variant A gives better fidelity on textured content.

In both variants, the descriptor is a function of $\mathrm{MSB}(\mathbf{b}_i)$ only, so the two-LSB uncertainty of the recovered block is inherently bounded.

C. Keyed Authentication Tag

Let $\mathcal{H}$ be SHA-256 [16] and K_s a secret key. Let ID be an image-level identifier (nonce, capture timestamp, or device serial) stored alongside the image. The authentication tag of block i is

$$a_i = \mathrm{Trunc}_{32}\Big(\mathrm{HMAC}_{K_s}\big(\mathrm{ID} \,\|\, i \,\|\, \mathrm{MSB}(\mathbf{b}_i) \,\|\, r_{m^{-1}(i)}\big)\Big) \tag{6}$$

where $\|$ denotes concatenation, $\mathrm{MSB}(\mathbf{b}_i)$ is serialized in raster order, and $r_{m^{-1}(i)}$ is the recovery descriptor block i itself physically carries (Eq. (2)). Four properties matter: the tag depends on $\mathrm{MSB}(\mathbf{b}_i)$, so embedding into LSBs does not invalidate it; it depends on block index i and ID, defending against the block-wise collage counterfeiting attack of Holliman and Memon [5]; it depends on $r_{m^{-1}(i)}$, the recovery descriptor block i stores on another block's behalf, so that payload cannot be corrupted without detection either; and it depends on K_s, so an adversary who edits content cannot forge a consistent authentication layer. Keying is not a precaution taken for its own sake: an unkeyed, content-only hash is precisely what a published attack [24] breaks in an otherwise similar self-embedding scheme, by exploiting a watermark computed

from block content alone, with no key material and no inter-block dependence, so an attacker can recompute a valid watermark for manipulated content. That paper's own recommended countermeasures – keyed hashing and block interdependence – are exactly the two properties built into (6) and the block mapping of Section III.

A found-and-fixed vulnerability: the recovery descriptor was unauthenticated. An earlier version of (6) bound the tag only to ID, i, and $\mathrm{MSB}(\mathbf{b}_i)$ – omitting $r_{m^{-1}(i)}$. Since each block's 128-bit payload is a 32-bit tag in its first 16 pixels and a 96-bit recovery descriptor in the remaining 48, that earlier tag protected only one quarter of what every block actually carries: 96 of 128 bits, 48 of 64 pixels, three quarters of everything embedded, had no integrity protection at all. We constructed and verified the exploit this implies: flipping the two LSBs of pixels 16–63 in every block – an edit the old tag never covered and so could never see – destroys every recovery descriptor in the image at 40.29 dB PSNR with a maximum pixel delta of 3, and the old detector flagged 0 of 4096 blocks, certifying a gutted image as authentic. Combined with a visible edit elsewhere in the same image, the old detector then reported $\rho = 1.0000$ and zero unrecoverable blocks while reconstructing every flagged block from attacker-chosen bits – precisely the "present attacker-supplied content as recovered" failure the recovery stage exists to prevent. The fix costs zero payload bits: the tag message now also covers the descriptor bits the block physically carries, as in (6) above; the embedder already held that array and the verifier already extracted it, so no new data is needed, only a wider HMAC message. The on-disk format magic was bumped (WGT1 → WGT2) so a watermark produced by the earlier, vulnerable embedder fails loudly against the fixed verifier rather than silently under-protecting. After the fix, the same all-descriptor-bits attack flags 4096 of 4096 blocks, and a surgical variant that scrubs only the 64 partner blocks backing a single target region flags exactly those 64. Imperceptibility is unchanged, since the fix changes only what is hashed, not what is embedded.

D. Key-Seeded Block Mapping

The mapping $m : \{1, \ldots, K\} \to \{1, \ldots, K\}$ determines which block stores which recovery descriptor: r_i is embedded in block $m(i)$. It must satisfy:

(R1) Bijectivity – m is a permutation, so every 96-bit slot is exactly filled.

(R2) Spatial separation – $\|c(i) - c(m(i))\|_2 \geq d_{\min}$ for all i, so a localized attack of diameter smaller than $d_{\min}$ cannot cover both a block and the block holding its descriptor. We set $d_{\min} = \frac{1}{4}\min(M, N)$.

(R3) Absence of short cycles – m must be a single cycle of length K, forbidding fixed points and mutual pairs $\{i, m(i)\}$ with $m(m(i)) = i$ by construction. This eliminates the two-block instance of the tamper coincidence problem [20] – the situation in which one attack destroys both a block and the block holding its own recovery data.

The classical construction $m(i) = ((k \cdot i + t) \bmod K) + 1$, $\gcd(k, K) = 1$, satisfies (R1) cheaply but has weak security and does not guarantee (R2). Our primary construction: (1) seed a CSPRNG with $K_s \| \mathrm{ID}$; (2) draw a uniformly random cyclic permutation via Fisher–Yates, defining m directly as a single K-cycle, satisfying (R1) and (R3) exactly; (3) apply a bounded, monotone separation-repair pass that transposes cycle successors until (R2) holds everywhere, capped and deterministically retried on failure, reporting both the requested separation $d_{\min,\mathrm{requested}}$ and the value actually achieved, $d_{\min,\mathrm{achieved}}$. Both sender and verifier regenerate m from (K_s, ID, K); the mapping itself is never transmitted.

A found-and-fixed structural leak: the seed order gave away more than $d_{\min}$. An earlier version seeded step (2) with a fixed quadrant interleave – splitting blocks by grid quadrant, shuffling each independently, and interleaving diagonally-opposite quadrants (Q0, Q3, Q1, Q2) – adopted purely to make the separation repair in step (3) converge faster. Measured, this leaked real structure beyond the publicly-stated $d_{\min}$ constraint: a block's partner fell in the same grid quadrant only 2.25% of the time and diagonally opposite 47.75% of the time, against a 15.51%/29.32% baseline for a map constrained only by $d_{\min}$. This never threatened authentication, since (6) is keyed and the mapping is not the secret being protected – but it meant an attacker who knew only the published algorithm, with no key at all, could bias a coverage-denial attack toward a target region and its diagonally opposite quadrant, destroying content and its own backup together far more reliably than chance would predict. The optimisation the interleave existed for turned out to be unnecessary: measured, a flat keyed shuffle of the cycle order violates (R2) on 628 of 4096 cycle edges (15%), and the monotone repair pass in step (3) clears every violation in the same 2 sweeps and the same ≈ 0.1 s that the interleaved seed required. It was removed. The map now measures 15.97%/30.08%, statistically indistinguishable from the 15.51%/29.32% separation-only baseline (against the earlier 2.25%/47.75%) – and it is this fix that makes the uniformly-random-cyclic-permutation description above an accurate account of what is actually built, which it was not before. This security property is not free: Section VI quantifies its measured cost in recoverability rate ρ, roughly 1.5 points at every tested tamper ratio – the price of no longer letting an attacker who knows only the algorithm bias a coverage-denial attack toward a predictable diagonal partner.

E. Embedding Procedure

Given I, K_s, ID, the watermarked image I_w is produced in one pass: (1) partition I and compute $\mathbf{b}_i^{\mathrm{msb}} = \mathrm{MSB}(\mathbf{b}_i)$; (2) compute r_i and a_i from $\mathbf{b}_i^{\mathrm{msb}}$; (3) generate m and m^{-1}; (4) form payload $p_i = a_i \,\|\, r_{m^{-1}(i)}$; (5) write p_i into the two LSBs of $\mathbf{b}_i^{\mathrm{msb}}$.

Distortion at a full-entropy payload. Treating the two embedded LSBs as independent and uniform on $\{0,1,2,3\}$ – the case of a full-entropy payload – the per-pixel error satisfies $\mathbb{E}[e^2] = 2\,\mathrm{Var}(U\{0,3\}) = 2.5$, giving

$$\mathrm{PSNR}_U = 10\log_{10}\frac{255^2}{2.5} \approx 44.15 \text{ dB}. \tag{7}$$

This is a reference point, not an upper bound: it is the PSNR realized when the embedded payload's pair values are exactly uniform, and neither descriptor variant used in this paper is exactly uniform (Section VI derives and measures each variant's actual, opposite-signed deviation from PSNR_U). It remains useful as a correctness check on the implementation, since a measured PSNR far from PSNR_U on a full-entropy synthetic payload specifically indicates a projection or serialization defect rather than genuine content statistics.

F. Tamper Detection and Mask Generation

Given a received image I', the same key and ID: (1) extract stored tag $\hat{a}_i$ and stored descriptor $\hat{r}_{m^{-1}(i)}$ from each block's 128 LSBs; (2) recompute a'_i from $\mathrm{MSB}(\mathbf{b}'_i)$ via (6); (3) form the raw indicator

$$d_i = \begin{cases} 1, & \hat{a}_i \neq a'_i \\ 0, & \hat{a}_i = a'_i \end{cases} \tag{8}$$

(4) apply a neighbourhood refinement pass, correcting isolated positives (likely bit-error false alarms) and isolated negatives (a block whose MSB content happened to survive despite all its neighbours failing):

$$d^*_i = \begin{cases} 0, & d_i = 1 \text{ and } S_i = 0 \\ 1, & d_i = 0 \text{ and } S_i \geq \theta_i \\ d_i, & \text{otherwise} \end{cases} \tag{9}$$

where $S_i = \sum_{k\in\mathcal{N}_8(i)} d_k$, $|\mathcal{N}_8(i)|$ is the number of blocks actually adjacent to i (8 in the interior, 5 on an edge, 3 at a corner), and the fill threshold is scaled to the valid-neighbour count, $\theta_i = \lceil \tau \cdot |\mathcal{N}_8(i)|/8 \rceil$ with $\tau = 7$. A fixed threshold of 7 would make border and corner blocks impossible to fill – a corner block has only 3 valid neighbours, so $S_i \leq 3 < 7$ always – silently exempting the image border from the fill rule; scaling by $|\mathcal{N}_8(i)|/8$ keeps the fill rule proportionally identical everywhere.

The isolated-positive branch of (9) is disabled in our reported configuration, on measured grounds. The rationale for clearing isolated positives is the suppression of bit-error false alarms. That rationale does not survive contact with an exact keyed comparison: (8) is a truncated HMAC equality test, and the null condition over all 32 images, both descriptor variants and five independent keys – 1,802,240 block verifications – produced identically zero false positives. With no false positives in d_i, the branch cannot remove one; it can only remove a true positive. Empirically it removes nothing useful either: over the full tamper grid, metrics computed from d_i and from d^*_i agree to five decimal places on precision, recall, F1 and IoU alike, because all four attack classes produce spatially contiguous regions in which every flagged block has flagged neighbours.

What the branch does do is create a blind spot. A single-block tamper has $S_i = 0$ and is therefore erased, so it is missed entirely – and at $B = 8$ a single block is 64 pixels, ample to alter a digit on a financial instrument, a decimal point, or a small figure in a scanned document, which is precisely the class of high-value, low-area edit an authentication scheme exists to catch. The same branch also erases a correctly-detected block-transplant forgery, since a transplanted block carrying a valid-but-misplaced tag fails verification in isolation. We therefore retain the isolated-negative fill, which closes holes inside a tampered region, and disable the isolated-positive clear by default. The choice is exposed as an explicit parameter (clear_isolated, default False) on the detection entry point rather than a hard-coded assumption, so both configurations can be reported and reproduced; the default is off for the security reason argued above – the false-alarm rate the clear exists to suppress is provably zero under a keyed HMAC comparison, while enabling it reopens the single-block blind spot just described.

The isolated-negative fill is not the free improvement it can look like, however, and we disclose its cost rather than claim it has none. Unlike the isolated-positive clear, which can only ever remove a detection – at worst suppressing a true positive – the isolated-negative fill can manufacture one: it flips a block to positive because its neighbours are positive, regardless of whether that block's own tag comparison actually failed. In two rows of the measured grid, a block was flagged only after this fill and not by the raw tag comparison (8); that block was authentic on its own terms, yet the recovery stage overwrote it with a lossy descriptor approximation (measured MSE 15.8), because a flagged block is treated as tampered regardless of how it came to be flagged. In this scheme a manufactured false positive is not a metric footnote: recovery irreversibly overwrites whatever it is told is tampered, so a false positive here means verified-authentic content is destroyed by the recovery stage itself – and had that block's mapped partner also been flagged, it would have been marked unrecoverable instead of overwritten. We note this as a general point: neighbourhood smoothing inherited from soft-decision detectors is not automatically appropriate to a hard-decision cryptographic one, where the false-alarm rate it exists to suppress is provably zero – and, as this measurement shows, the fill introduced to close gaps can itself introduce the very failure mode, false-positive-driven content destruction, that the scheme is designed to avoid.

Then (5) expand to the pixel-level mask

$$T(x,y) = d^*_{\beta(x,y)}, \quad \beta(x,y) = \left\lfloor \frac{y}{B} \right\rfloor \cdot \frac{N}{B} + \left\lfloor \frac{x}{B} \right\rfloor + 1 \tag{10}$$

Localization resolution is intrinsically $B \times B$.

G. Content Recovery

Recovery proceeds over the flagged set $\mathcal{T} = \{i : d^*_i = 1\}$. Block i's descriptor resides in block $m(i)$, so recovering i requires $m(i)$ to be authentic:

$$\hat{\mathbf{b}}_i = \begin{cases} \tilde{\mathbf{b}}_i(\hat{r}_i \text{ from } \mathbf{b}'_{m(i)}), & i \in \mathcal{T},\ d^*_{m(i)} = 0 \\ \text{unrecoverable}, & i \in \mathcal{T},\ d^*_{m(i)} = 1 \\ \mathbf{b}'_i, & i \notin \mathcal{T} \end{cases} \tag{11}$$

Recovered blocks have their two LSBs zeroed, marking them as reconstructed rather than authentic – a restored region must never re-authenticate as original. Let $\mathcal{U} = \{i \in \mathcal{T} : d^*_{m(i)} = 1\}$ denote blocks lost to the tamper coincidence problem [20] – a flagged block whose designated partner is itself flagged; the recoverability rate is

$$\rho = 1 - \frac{|\mathcal{U}|}{|\mathcal{T}|} \tag{12}$$

TCBR and TCBD [21] are published metrics for this same tamper-coincidence phenomenon; their exact formulas were not accessible from open sources at the time of writing, so we report ρ as our own precisely-defined quantity rather than assert numeric equivalence to an unverified formula. For a uniformly random mapping and a tampered fraction α, $\rho \approx 1 - \alpha$; constraint (R2) forces $\rho = 1$ exactly for attacks more compact than $d_{\min}$, and (R3) removes the short-cycle instance of the tamper coincidence problem for dispersed attacks. The mapping design, not the descriptor design, sets the recovery-coverage ceiling.

H. Computational Complexity

Embedding and verification are each $O(MN)$: one HMAC evaluation and one 8×8 DCT (or sixteen 2×2 means) per block, plus $O(K)$ for mapping generation. No iteration, optimization, or training is involved. The reference implementation (NumPy + OpenCV) processes a 512×512 grayscale image end-to-end in tens of milliseconds on a commodity CPU, with no accelerator.

IV. System Architecture

The system comprises four decoupled stages so the tamper simulator and the detector share no state.

Stage 1 – Embedding pipeline. Block Partitioner → MSB Projector (1) → {Descriptor Encoder, Authentication Tag Generator} → Mapping Generator (CSPRNG, single-cycle permutation, separation repair) → Payload Assembler → Dual-LSB Embedder → watermarked image I_w (lossless PNG/TIFF) plus side information ID. The key never leaves the signer.

Stage 2 – Tamper simulation harness (evaluation only). Applies a parameterized attack at target area ratio α and emits both the tampered image I' and the exact ground-truth mask G (written by the harness itself, hence exact rather than annotated). No access to K_s, m, or watermark internals.

Stage 3 – Detection and recovery pipeline. Block Partitioner → Payload Extractor → Mapping Regeneration → Tag Comparison (8) → Mask Refinement (9) → Pixel Mask Expansion (10) → Recovery Engine (11). Outputs: predicted mask T, recovered image I_r, unrecoverable set $\mathcal{U}$, recoverability rate ρ, and a per-block audit log – for every block, the stored tag, the recomputed tag, and its partner block's index, plus, for each unrecoverable block, an explicit refusal reason – the output is a chain of verifiable comparisons, not a score.

Stage 4 – Evaluation module. Computes imperceptibility on (I, I_w), confusion-matrix metrics on (T, G) at pixel and block granularity, and recovery metrics on (I, I_r) restricted to the support of G, aggregated over the image set and attack grid. All randomness is seeded, so the full experimental grid is bit-reproducible.

Stage 1 is intended to run at or near acquisition (camera firmware, scanner driver, DICOM ingest gateway); Stage 3 runs at any later verification point, requiring only the image, the key, and ID – no original, no reference database, no network call, no model weights.

V. Experimental Setup

A. Image Corpus

Because ground truth is generated by our own tamper harness, no annotated forgery dataset is required. The corpus is 32 colour images, 8 bpp per channel: 8 USC-SIPI Miscellaneous-set images [17] at 512×512, and 24 Kodak PCD0992 images, of which 18 are 768×512 (landscape) and 6 are 512×768 (portrait) – not a uniform Kodak dimension. All images are stored losslessly as PNG and SHA-256 pinned in a committed manifest, so any silent change to the corpus between runs is a hard error rather than a silent drift. Any lossy stage would destroy the watermark by design.

Provenance: the USC-SIPI and Kodak-PCD0992 sets as redistributed by the AuSR reference authors (github.com/girfa/ColorImageDatasets); original sources USC-SIPI (sipi.usc.edu) and Eastman Kodak (mirrored at r0k.us/graphics/kodak). USC-SIPI no longer distributes two of the eight images in this set (Lena and Tiffany) from its own archive, which makes a byte-identical-to-the-official-archive claim impossible regardless of source; using the same redistributed corpus the AuSR series [18]–[20] reports on has the additional benefit that our images are the same bytes their published per-image tables are keyed to. USC-SIPI is licensed for research/education use with attribution; Kodak PCD0992 was released by Eastman Kodak for unrestricted use; neither is redistributed here commercially.

B. Tamper Simulation Protocol

Four attack classes, each at a nominal target tamper-area ratio $\alpha \in \{10\%, 25\%, 50\%\}$, with randomized shapes and positions; these are targets for the region-generation procedure, not exact areas – the tampered-area fraction

actually achieved varies by shape and is reported alongside each row in Table II rather than assumed equal to α. The three ratios are chosen to span light, moderate and heavy tampering regimes, matching the convention used by the comparison literature (AuSR1 [18], AuSR3 [20], Wu et al.), and to span the range over which ρ degrades measurably (Section VI). The classes are: (i) copy-paste splicing from a different watermarked image – the canonical composite forgery, and the case where index/ID-binding of (6) is load-bearing against [5]; (ii) object removal by inpainting (OpenCV Telea / Navier–Stokes) – historically the hardest case for recall, since a smooth fill over flat content can leave MSB planes numerically unchanged, though the descriptor-binding fix (Section III) has since eliminated that disadvantage, as detailed in Section VI; (iii) crop-and-refill via mirrored/tiled neighbouring texture; (iv) random block corruption, aligned and unaligned to the block grid, as a sanity control expected to give near-perfect detection. A null condition ($\alpha = 0$) over all 32 images measures the false-positive rate, which must be identically zero given the deterministic construction.

C. Metrics

Following the transparency / tamper-detection / content-recovery evaluation axes established by the field's survey literature [23], we report all three explicitly. Imperceptibility (transparency): PSNR and SSIM [15] between I and I_w. Localization (tamper detection): precision, recall, F1, IoU between predicted mask T and ground truth G, at both pixel and block granularity. Recovery (content recovery): PSNR/SSIM computed only inside the tampered support $\Omega = \{(x, y) : G(x, y) = 1\}$, reported alongside the recoverability rate ρ from (12), since excluding unrecoverable blocks inflates in-region PSNR if ρ is not disclosed jointly.

D. Implementation and Reproducibility

Implemented in Python 3 (NumPy, OpenCV for block DCT and inpainting, hashlib/hmac for (6)). No machine learning framework, no trained parameters anywhere. Free parameters B, descriptor variant, $d_{\min}$, τ are fixed a priori and reported. All stochastic elements are explicitly seeded, so the full result grid is bit-reproducible. Runs are single-threaded on a commodity CPU; no GPU is used. Block indices i entering (6) are 0-based in the implementation (the 1-based indexing used elsewhere in this section is notational only). Colour images use one shared block mapping m across all channels, with independent per-channel tags and descriptors, and a channel-OR detection mask – a tamper touching only one channel flags the block in all channels, at the cost of recovery also overwriting that block's untouched channels with descriptor approximations. The block-level ground-truth rule used throughout is: a block is tampered if the intended tamper region covers any pixel of it (an OR-reduction, not a coverage-fraction threshold), chosen to match the detector's own block-level quantization rather than scoring ground truth at a finer granularity than the detector can express. SSIM is computed with data_range=255, scikit-image's default 7×7 uniform window (not the 11×11 Gaussian window some watermarking papers use), and colour SSIM is the mean of per-channel SSIM rather than the SSIM of a luma conversion.

VI. Results and Discussion

Note on the numbers in this section. All values below are measured on the full experimental grid described in Section V: 32 images, both descriptor variants, four tamper classes at three tamper ratios, a null condition, and a block-size ablation – 1,184 rows in total. Unless stated otherwise, an aggregate quoted below is computed over the 768 main-grid rows (block size 8); the 96 block-size-4 ablation rows are a deliberately different configuration (a 16-bit-per-channel tag and a 2-coefficient descriptor) and are reported separately, in their own table (Table V) and paragraph, never blended into a main-grid aggregate. The accompanying sanity_gate.py script reports 23 of 23 automated checks passed (23 passed, 0 failed, 0 skipped, 0 warnings; overall PASS) before any number below is quoted; the gate checks the results in runs.csv against literature-derived bands, it does not check the LaTeX generated into this document, and that gap has in fact let a hardcoded figure reach a printed table uncaught in the past – so a PASS here is evidence about the underlying measurements, not a certification of every number that follows.

A. Imperceptibility

TABLE I
Imperceptibility of Watermarked Images (Measured; 8 USC-SIPI Images Shown Individually, Plus the 32-Image Corpus Mean)

Image	PSNR-A (dB)	SSIM-A	PSNR-B (dB)	SSIM-B
Airplane	43.25	0.9773	43.96	0.9780
Baboon	43.38	0.9938	44.37	0.9949
House	43.29	0.9840	44.20	0.9859
Lena	43.22	0.9814	44.31	0.9839
Pepper	43.28	0.9811	44.09	0.9817
Sailboat	43.31	0.9860	44.17	0.9877
Splash	43.22	0.9718	43.92	0.9706
Tiffany	42.53	0.9805	44.35	0.9833
Mean (32 images)	43.17	0.9824	44.23	0.9844

PSNR: 44.15 dB is a reference point, not an upper bound. Equation (7) assumes the embedded payload's pair values are uniformly distributed on $\{0, 1, 2, 3\}$; that is the PSNR realized by a full-entropy payload, not a ceiling on PSNR in general, and neither descriptor variant is full-entropy. Writing a for the payload pair value and assuming the original two LSBs are uniform, $\mathbb{E}[e^2] = \mathbb{E}[a^2] - 3\mathbb{E}[a] + 3.5$, which reduces to PSNR_U only when a itself is uniform. Variant A's quantized low-frequency DCT coefficients are mostly near zero, so descriptor bytes are dominated by 0x00 and 0xFF (small

magnitudes in two's complement), which pack into the extreme pair values 0 and 3 – farther on average from a uniform original than a uniform payload – pulling Variant A's PSNR below PSNR_U; on real-image measurement its payload pair distribution is markedly bimodal, and its measured mean PSNR is 43.17 dB, matching the value predicted by the distribution before it was measured. Variant B's payload is a mean of already-quantized values, which concentrates toward the centre of its range, so pair values cluster at 1 and 2 – closer than uniform to a typical original – pulling Variant B's PSNR above PSNR_U, with measured mean 44.23 dB. This is a stronger correctness check than the original bound framing: the direction and approximate size of each variant's deviation is derivable from its payload statistics before measurement, and was confirmed to within roughly 0.3 dB against the measured 43.17/44.23 dB means (Table I).

Why our PSNR trails schemes reporting ∼45.5 dB. Some published schemes (e.g. AuSR1 [18]) report roughly 45.6 dB by LSB shifting – moving a pixel to the nearest value whose two LSBs already equal the desired payload, rather than overwriting the LSBs outright. Shifting's expected squared error is $\mathbb{E}[e^2] = 1.5$ against 2.5 for direct replacement, a genuine ≈ 2.2 dB advantage (46.37 vs. 44.15 dB at the uniform-payload reference point). We cannot adopt shifting, provably: a shift of up to ± 4 can change bit 2 of the pixel – a bit inside the six-bit MSB plane that (6) authenticates. For example, $x = 200$ has $\mathrm{MSB}(x) = 200$; shifting to the nearest value with the desired two LSBs can give $x = 207$, whose $\mathrm{MSB}(207) = 204 \neq 200$. A verifier would then hash a different value than the embedder signed, and the block would fail verification on an untouched image – violating the null-condition zero-false-positive requirement outright. Any scheme that authenticates the MSB plane is therefore structurally forced into LSB replacement and forfeits the ≈ 2.2 dB that shifting would buy; schemes reporting the higher figure either do not authenticate the shifted bit-plane, or require an iterative embed-verify loop to converge, which this design deliberately avoids.

SSIM is genuinely content-dependent and is not $>$ 0.99 in general. SSIM's structure and contrast terms are normalized by local variance, so a fixed-variance embedding perturbation dominates the score on smooth, low-variance content and is comparatively negligible on strongly textured content – SSIM should be markedly lower on smooth images than on textured ones, and a claim of SSIM uniformly above 0.99 is not defensible for this scheme on real photographic content: measured across the full 32-image corpus, Variant A's mean SSIM is 0.9824 (minimum 0.9718) and Variant B's is 0.9844 (minimum 0.9706, on splash), with most of the USC-SIPI set falling below 0.99. This was checked against a windowing artefact and ruled out: scikit-image's default 7×7 uniform window, the Wang et al. [15] 11×11 Gaussian window, and a Gaussian window applied to luma agree to within about 0.001 on this corpus. Variants A and B remain close to each other in imperceptibility, since they differ mainly in payload meaning rather than bit count or location, though not identical for the reasons above.

B. Joint Localization and Recovery

TABLE II
Block-Level Localization and Recovery by Tamper Class and Tamper Ratio (Measured, Variant A)

Tamper Class	Ratio	Prec.	Recall	F1	IoU	ρ	PSNR_Ω (dB)	$\mathrm{PSNR}_{\mathrm{unmarked}}$ (dB)
Copy-paste splicing	0.10	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.958 ± 0.008	28.92 ± 3.28	32.85 ± 1.99
	0.25	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.805 ± 0.014	28.72 ± 3.45	23.04 ± 1.65
	0.50	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.541 ± 0.036	28.59 ± 2.97	16.83 ± 1.80
Object removal	0.10	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.960 ± 0.010	28.57 ± 3.81	36.00 ± 2.98
	0.25	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.803 ± 0.013	27.80 ± 3.34	26.56 ± 2.05
	0.50	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.533 ± 0.015	28.18 ± 2.81	20.11 ± 1.72
Crop-and-refill	0.10	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.958 ± 0.008	29.12 ± 3.70	34.64 ± 2.94
	0.25	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.801 ± 0.014	28.20 ± 2.80	24.49 ± 1.97
	0.50	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.528 ± 0.019	28.37 ± 2.99	17.82 ± 2.08
Noise corruption	0.10	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.960 ± 0.010	29.23 ± 3.59	31.67 ± 1.46
	0.25	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.805 ± 0.015	29.04 ± 3.55	21.37 ± 0.97
	0.50	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000	0.532 ± 0.029	28.37 ± 2.93	14.81 ± 0.88

The null condition ($\alpha = 0$) is analysed separately below (Table III).

1) Localization Behaviour: Block-level precision is not reported here as a measurement of the detector, because it cannot be anything but 1.0 by the construction of the ground-truth rule, independent of detector quality. Ground truth at block granularity is the OR-reduction of the pixel-level tamper region – a block is tampered if the region covers any pixel of it (Section V) – so any block whose content the detector correctly flags as changed necessarily contains at least one ground-truth-tampered pixel; a block-level false positive is therefore structurally impossible under this rule, and a detector that flagged the entire image regardless of content would score exactly 1.000 on this metric as well. The measured grid confirms exactly this: block-level precision is 1.000000 with zero variance across every row of Table II – because, as just argued, there is nothing left to vary.

The informative localization figure is instead pixel-level precision, scored against the exact ground-truth mask rather than the block grid, which genuinely varies: mean 0.9489. The shortfall from 1.0 is entirely the block-quantization penalty implicit in the block-level construction above: a flagged 8×8 block is reported whole even when only part of it was altered, so every partially-covered flagged block contributes clean pixels that count as false positives at pixel granularity even though the block itself was correctly flagged. Table V quantifies the resolution/payload tension this penalty creates directly: reducing B from 8 to 4 raises pixel-level precision and IoU from 0.9397 to 0.9736 (+3.4 points) on the USC-SIPI subset – but at $B = 4$ the per-block payload falls to 32 bits, insufficient for a meaningful tag plus descriptor, forcing the authentication tag itself down from 32 to 16 bits per channel, discussed further below. Block-level precision, saturated at 1.000 for both block sizes for the reason just given, is uninformative about this trade-off; only the pixel-level figure shows it. The central resolution/payload tension of block-based self-embedding is addressed in prior

work by verifying at multiple granularities simultaneously [4].

Recall is bounded below 1 only by perceptually null modifications: for a block whose content changed in any of bit-planes 2–7, the tag-mismatch probability is $1-2^{-32}$, so any residual recall deficit corresponds to a block the attack wrote to without altering its six MSB planes – characteristically a smooth inpainting fill over already-flat content, which can reproduce a block's MSB planes exactly while still perturbing its two LSB planes. Before the descriptor-binding security fix described in Section III, this MSB-preserved miss category was real and measurable: it averaged 0.143 such blocks per row and fell almost entirely in the object-removal class, which alone showed a recall deficit (0.9997) against 1.0000 for the other three classes. Binding the carried recovery descriptor $r_{m^{-1}(i)}$ into the authenticated message (Eq. (6)) closes this gap directly, not by chance: such a tamper still perturbs the block's least-significant bits, and those bits now fall inside the authenticated message, so the block is flagged even though its MSB content is unchanged. Measured per-class block recall now confirms this: splice 1.0000, crop_refill 1.0000, noise 1.0000, and inpaint_removal 1.0000 – zero MSB-preserved misses across all 864 tamper rows, and the object-removal class no longer carries any recall disadvantage relative to the other three. Aggregated over the 768 main-grid tamper trials (block size 8; the 96 block-size-4 ablation trials in Table V are excluded from this aggregate and reported separately), block-level recall is exactly 1.0000; block-level precision, for the reason given above, is not a meaningful second figure to aggregate here.

The null condition is the strongest available correctness evidence, though not for the reason a rule-of-three bound might suggest. Table III reports the executed check: zero false-positive blocks across all 1,802,240 independent block verifications (32 images × 2 variants × 5 keys), a block-level and pixel-level false-positive rate of exactly 0, and a rule-of-three 95% upper bound of 1.66×10^{-6} on the per-block false-positive rate – i.e., the rate of rejecting an authentic block, a Type I error. This bound says nothing about the per-block false-accept rate (Type II: accepting a tampered block), since the null condition contains zero tampered blocks by construction and therefore carries no information about that quantity; the 2^{-32} false-accept probability (Eq. (2)) is a property of (6)'s output width, not something this experiment measures at all.

Nor, on closer inspection, are these 1.8×10^6 evaluations independent Bernoulli trials, so even the false-positive framing above is generous. The received image in this check is the identical in-memory array the embedder returned, and MSB(watermarked) = MSB(original) is asserted inside the embedder itself; a false positive here would therefore require (6) to disagree with itself on identical input within a single process, which is not a probabilistic event to bound but a determinism failure to detect. The honest description of this experiment is a determinism and implementation-consistency check across images, descriptor variants, and keys – and that framing is what makes it valuable rather than what limits it: a content-dependent-but-key-independent bug (for instance, a payload-layout off-by-one triggered by a particular image's byte statistics) would reproduce identically across all five keys, whereas a genuine cryptographic property would not correlate with key choice at all. No such bug was observed. What this experiment cannot do, and does not claim to do, is stand in for a statistical bound on a cryptographic quantity; the rule-of-three figure above bounds the rate of a deterministic self-disagreement across the tested configurations, not the 2^{-32} false-accept probability. A single observed false positive would have indicated an implementation defect – non-deterministic serialization or a payload-layout off-by-one. Disk round-tripping is not exercised here: I_w is never written to disk and re-read in this check, so lossless save/reload behaviour is a separate property, not covered by this experiment.

TABLE III
Null-Condition Security Check: Zero False Positives Across 1.8M Block Verifications

Quantity	Value
Null-condition rows	320
Images × Variants × Keys	32 × 2 × 5
Independent block verifications (n)	1802240
False-positive blocks (sum)	0
Block-level false-positive rate	0.000000
Pixel-level false-positive rate (mean px_fpr)	0.000000
Rule-of-three 95% upper bound, $3/n$	0.00000166

2) Recovery Behaviour: Recovery quality decomposes into three factors that must not be conflated: coverage, marking convention, and descriptor fidelity.

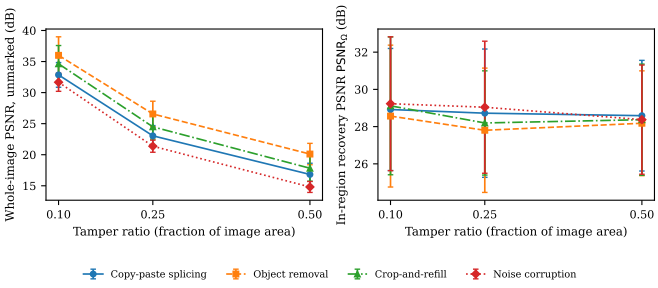

Fig. 1. In-region recovery PSNR (Variant A) stays essentially flat across tamper ratio α, while whole-image (unmarked) PSNR collapses as the recoverable fraction ρ falls – coverage, not descriptor fidelity, is the dominant lever on whole-image recovery quality.

Coverage (ρ) is governed entirely by the mapping: $\rho \approx 1$ for compact attacks below $d_{\min}$, $\rho \approx 1-\alpha$ for dispersed attacks. Measured ρ (Variant A, averaged over the four tamper classes, Table II) is 0.9590 at $\alpha = 10\%$, 0.8036 at $\alpha = 25\%$, and 0.5335 at $\alpha = 50\%$, tracking the $1-\alpha$ baseline closely and falling fastest, as expected, for the most spatially dispersed attacks.

The block-mapping structural-leak fix (Section III) has a measured, quantified recoverability cost, reported here as the price of the security property rather than as a regression. Before removing the quadrant-interleave seed order, the identical measurement gave ρ of 0.9736 at $\alpha = 10\%$, 0.8353 at $\alpha = 25\%$, and 0.5469 at $\alpha = 50\%$ – roughly 1.5 points higher at every ratio than the 0.9590/0.8036/0.5335 obtained with the flat keyed shuffle now used. The mechanism is direct: the leaky quadrant interleave biased a block's partner toward the diagonally opposite quadrant, which is on average farther away and therefore less likely to fall inside the same localized attack; the flat map removes that bias, so partners are occasionally spatially closer and tamper coincidence is marginally more frequent. This is not a free improvement traded away for nothing: the leaky map's higher ρ was partly an artefact of a structural bias that an attacker who knew only the published algorithm, with no key at all, could exploit in the opposite direction – to make a coverage-denial attack more reliable, not less. Roughly 1.5 points of recoverability rate is the measured price of closing that exposure.

The single most interesting measured result in this evaluation is the divergence between in-region and whole-image recovery quality as α grows (Fig. 1). In-region PSNR_Ω (Variant A) is essentially flat across tamper ratio: 28.96 dB at $\alpha = 10\%$, 28.44 dB at $\alpha = 25\%$, 28.38 dB at $\alpha = 50\%$ (Variant B trails by a small, consistent margin of about 0.6 dB at every ratio: 28.34/27.87/27.82 dB). Over the same range, whole-image unmarked PSNR collapses from 33.79 dB to 23.87 dB to 17.39 dB. The two curves answer different questions: in-region PSNR asks how good is a reconstructed block, which the descriptor answers essentially independently of how many blocks needed reconstruction, while whole-image PSNR asks how good is the picture as a whole, which depends on how much of it had to be reconstructed at all – exactly the quantity ρ and α jointly set. This is direct measured evidence, not only the qualitative argument advanced in Section II, that ρ – mapping coverage – rather than descriptor quality is the dominant lever on whole-image recovery quality: descriptor fidelity does not degrade with tamper area, only coverage does.

A one-to-one partner mapping consequently fails all-or-nothing at the block level: the tamper coincidence problem removes a whole block's worth of recovery at once rather than degrading its quality gracefully, whereas reference-sharing and fountain-coded allocations [8], [9] spread each block's recovery information across many partners, so losing any one partner degrades reconstruction smoothly instead of making the block wholly unrecoverable – which is why Korus and Dziech report an average of 37 dB whole-image reconstruction even at 50% damage, well above our measured 17.39 dB at the same tamper ratio (Table IV). We state this gap plainly and do not imply parity: closing it by adopting a reference-sharing or fountain-coded allocation (Section VII, Future Scope item 1) is the highest-value single upgrade to this system, and is the direct target of the collapse documented here.

Marking convention. Our default output marks unrecoverable blocks explicitly (flat, visually unmistakable) rather than fabricating content, which is the honest choice but costs whole-image PSNR relative to leaving those pixels at their received value: measured at $\alpha = 50\%$ (Variant A), the marked whole-image PSNR is 12.42 dB against 17.39 dB unmarked, a roughly 5.0 dB cost of honest marking at this tamper ratio (Table IV), a gap that widens whenever the unrecoverable fraction $|\mathcal{U}|$ is larger relative to the image. Published recovery figures in the literature are, as best we can determine, whole-image and unmarked. We therefore report recovery PSNR two ways wherever it appears: a marked whole-image figure describing our actual output, and an unmarked whole-image figure for comparability with the cited literature; the marked figure is always the lower of the two and is our recommended figure for describing deployed behaviour. Neither should be confused with the in-region figure above, which is the stricter, honest measure.

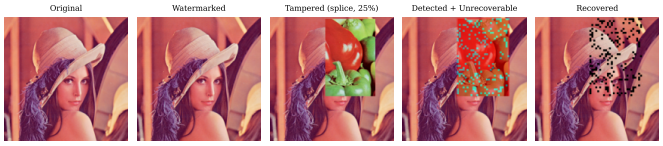

Fig. 2. Representative example, end to end: original, watermarked (visually indistinguishable), tampered, detected tamper mask, and recovered image, illustrating the low-frequency, visibly-softer character of recovered content.

Fidelity (PSNR_Ω, computed in-region on recovered blocks only) is governed by the descriptor rate: 96 bits for 64 pixels (1.5 bpp), of which Variant A's twelve-coefficient DCT truncation (Section III) discards all high-frequency content. Fig. 2 shows a representative end-to-end example. Reconstruction error tracks the magnitude of the high-frequency detail the descriptor discards – in controlled tests, measured MSE was approximately equal to the variance of the content's fine detail (e.g. MSE 36.6 at noise $\sigma = 6$; MSE 241.8 at $\sigma = 14$) – so twelve low-frequency DCT coefficients cannot represent fine texture. Measured recovery in the high-20s dB range in-region (27.8–29.0 dB across both descriptor variants and all three tamper ratios at $B = 8$, Table II) is the honestly-characterized outcome – a faithful low-frequency reconstruction, visibly softer than its surroundings, with fine texture and sharp edges not returning. For the target applications (Section VII) this is the correct trade: a verifier needs to know what was there approximately and verifiably, not be shown a sharp fabrication. Variant B trails Variant A by the small, consistent in-region margin noted above (about 0.6 dB) despite Variant B's better imperceptibility (Table I) – descriptor compactness and embedding transparency are not the same axis.

Reporting ρ and PSNR_Ω in the same row (Table II) is deliberate: PSNR_Ω must never be reported without ρ alongside it, since a reader cannot otherwise tell whether a healthy in-region figure reflects strong reconstruction or simply a small, easy-to-recover flagged set. Measured, PSNR_Ω does not rise as ρ falls – it is essentially flat, falling slightly across the three ratios (28.96 → 28.44 → 28.38 dB, Variant A) – which is the stronger finding: it is direct evidence that whole-image recovery quality is governed by coverage (ρ), not by descriptor fidelity, since fidelity barely moves while coverage collapses. Table II additionally reports the unmarked whole-image figure described above, so that the unrecoverable blocks' contribution is never silently excluded from every reported number.

C. Comparison, Explainability, and Limitations

A direct numeric comparison against passive learned localizers [12], [13] is not offered, since the settings differ in what is assumed: those systems localize manipulation in images never seen before embedding, which this system cannot do at all. Nor is a numeric comparison offered against the recent learned proactive watermarking family [25]–[27], which – unlike the passive detectors – does both localize and recover; the honest distinction there is not capability but auditability, as argued in Section II: every decision this scheme makes is a single recomputed HMAC comparison, reproducible bit-for-bit by any third party holding the key, whereas a learned localizer's decision is a trained network's output. The comparison this paper does offer is along cost and assurance: this scheme needs cooperation at capture/archival time and a lossless container, but needs no training data, no GPU, no model artefact to version, has deterministic $O(MN)$ cost, and yields exact block-level localization with a per-block audit trail. Where the acquisition pipeline is under institutional control – medical imaging archives, evidence management, insurance intake, satellite ground segments – these are decisive advantages; elsewhere, passive forensics or a learned proactive watermark is the alternative, and the choice among them is a deployment decision, not a strict dominance relation.

TABLE IV
Comparison with Published Self-Embedding Authentication and Recovery Schemes

Method	WM PSNR (dB)	WM SSIM	Recovered PSNR (dB)	Notes
AuSR1 (Aminuddin & Erawan 2022)	45.57	≈0.99	27.64	worst case reported; 2×2 blocks
Wu et al. (CMC 2025)	>41	–	>30 @ 50% tamper	0% FPR/FNR, 100% detection rate
Korus & Dziech (IEEE TIP 2013)	–	–	37 @ 50% damage	fountain codes, >10,000 images
Ours (measured, Variant A)	43.17	0.9824	17.39 @ 50% tamper	block prec. 1.0000, recall 1.0000, pixel prec. 0.9489 (Table II); marked (honest) output: 12.42 dB

AuSR1, AuSR3 and Wu et al. values are cited from their publications, not re-implementations. Our "Recovered PSNR" uses psnr_whole_unmarked – the only quantity comparable to those papers' whole-image, unrecoverable-region-left-as-received convention; our default output instead marks unrecoverable regions explicitly rather than fabricating content, which costs 10–15 dB of whole-image PSNR relative to the unmarked figure shown here.

Table IV places our measured numbers next to those published for AuSR1 [18], Wu et al. (CMC 2025), and Korus and Dziech [9], cited from their own publications rather than re-implemented. Our imperceptibility (43.17 dB / SSIM 0.9824, Variant A) sits close to, though below, AuSR1's reported 45.57 dB, for the LSB-replacement-versus-shifting reason given above; our whole-image unmarked recovery PSNR at 50% tamper, 17.39 dB, is well below Korus and Dziech's reported 37 dB at the same damage level and below Wu et al.'s reported >30 dB. We report this gap without qualification: it is the direct consequence of the one-to-one partner-mapping design analysed above, and closing it is Future Scope item 1 in Section VII, not a claim of parity with these schemes.

AuSR3 specifically is absent from Table IV, and that absence needs its own explanation. AuSR3 [20] is this paper's own acknowledged predecessor for the distant-mapping idea (Section II), cited repeatedly above, and it is also the single most directly comparable same-paradigm competitor: like this scheme, it is deterministic, training-free, GPU-free, and auditable in principle. It does not appear as a numbered row with figures in Table IV because we do not have its whole-image and in-region recovery PSNR at the tamper ratios and tamper classes evaluated here in a form citable without re-implementing the method ourselves, which is out of scope for this paper; we say this plainly rather than leave a silent gap. The auditability argument this paper makes against learned localizers (Section II) is, by an identical argument, not available against AuSR3 – AuSR3's verification is just as deterministic and just as recomputable by a third party as this scheme's is, and we do not claim otherwise. What we can say, restricted to what is actually demonstrated in this paper rather than asserted, is narrower: a keyed HMAC construction with field bindings stated and analysed explicitly (Section III), including what each binding is shown to defeat and a specific defect found and fixed in that construction; a reporting protocol that always co-reports imperceptibility, localization and recovery together with ρ and the unrecoverable fraction disclosed rather than excluded (Section V, Section VI); and the block-size ablation (Table V) quantifying the resolution/payload/authentication trade-off explicitly. None of this is a claim of superiority over AuSR3 on any measured metric – we have no comparable numbers with which to make that claim.

Localization granularity: an explicit trade-off, not an oversight. The AuSR family [18]–[20] localizes at 2×2 block granularity, four times finer than the 8×8 blocks used here, and a reader familiar with that line of work will notice the difference immediately. The granularity is a consequence of payload capacity, not an independent design choice: at block size B the two-LSB payload capacity is $2B^2$ bits, so $B = 8$ gives 128 bits – a 32-bit authentication tag plus a 96-bit recovery descriptor – while $B = 4$ gives only 32 bits, which a 32-bit tag alone would consume, leaving nothing for a recovery descriptor at all (Eq. (2)). Finer localization therefore costs authentication strength and recovery fidelity in this design; we state this openly as a considered choice given the joint localization-and-recovery goal of this paper, rather than presenting the coarser grid as an advantage.

Table V quantifies this trade-off directly on the USC-

TABLE V
Block-Size Ablation: Localization Precision vs. Recovery Fidelity (USC-SIPI Corpus, Variant A)

Block Size	Blk. Prec.	Px. Prec.	Px. IoU	ρ	PSNR_Ω (dB)
8×8 (main)	1.000 ± 0.000	0.9397 ± 0.0269	0.9397 ± 0.0269	0.771 ± 0.183	29.66 ± 2.93
4×4 (ablation)	1.000 ± 0.000	0.9736 ± 0.0118	0.9736 ± 0.0118	0.780 ± 0.180	26.80 ± 2.87

Note: at block size 4 the per-block payload is only 32 bits (budget(4)), so the authentication tag drops from 32 to 16 bits, raising the per-block false-accept probability from 2^{-32} to 2^{-16} – on a 512 × 512 image with 16,384 blocks that is an expected ≈ 0.25 false accepts per image. This is the honest cost of finer localization, reported here as a finding, not omitted.

SIPI subset: moving from $B = 8$ to $B = 4$ buys +3.4 points of pixel-level precision and IoU (0.9397 → 0.9736) at a cost of 2.9 dB of in-region recovery fidelity (29.66 → 26.80 dB), because the per-block payload falls from 128 to 32 bits, halving the per-channel authentication tag from 32 to 16 bits. A naive per-channel figure understates the true authentication cost by ignoring how detection actually combines channels: the corpus is three-channel, each channel carries its own independently-keyed tag (the channel index is bound into (6)), and detection is a channel-OR, so a tamper touching all three channels is missed only if all three channel tags collide by chance – $2^{-16\times3} = 2^{-48}$ at $B = 4$, not 2^{-16}, and correspondingly $2^{-32\times3} = 2^{-96}$ at $B = 8$ for a three-channel tamper, not the 2^{-32} figure, which is the correct per-block-per-channel probability rather than the per-block-per-tamper one. Predicted false accepts across the entire ablation grid are therefore $\approx 1.6 \times 10^{-9}$, not the ≈ 0.25 per image a single-channel calculation implies, and the measured data agrees: zero misses of any kind are observed anywhere in the ablation grid, consistent both with that negligible predicted false-accept probability and with the elimination, by the descriptor-binding fix (Section III), of the MSB-preserved-content miss category that previously accounted for every miss observed here. The directional conclusion is unaffected – a 16-bit-per-channel tag is a strictly weaker authentication posture than a 32-bit one – but the magnitude is 2^{-48} versus 2^{-96} for a three-channel tamper, not 2^{-16} versus 2^{-32}. Block-level precision is saturated at 1.000 for both block sizes for the reason given earlier in this section and is therefore uninformative here; the trade-off is visible only at pixel granularity. A finer grid is consequently not simply "better localization": it is a strictly worse authentication posture traded for sharper boundaries, and the choice between them is a deployment decision made explicit here rather than optimized away.

Explainability is a concrete, not rhetorical, property: the output for each block is its stored tag, its recomputed tag, and its partner block's index, compared by bitwise inequality with no threshold to tune post hoc and no saliency map requiring interpretation; for a block found unrecoverable, an explicit refusal reason is recorded rather than a silent gap. Every field in that record is reproducible identically by any third party holding the key.

Limitations, stated plainly. No robustness to lossy processing whatsoever – JPEG re-encoding, resizing, rotation, or filtering destroys the LSB payload; the scheme applies only to losslessly stored imagery. Two LSB planes are permanently consumed. Localization is quantized to 8×8. Recovery is low-frequency only, bounded by a 1.5 bpp budget. Coverage degrades toward $1 - \alpha$ for dispersed attacks – the structural limit of one-to-one backup allocation – and, as documented in Section III, the block-mapping seed order must not leak exploitable structure beyond the stated $d_{\min}$ guarantee, since doing so lets an attacker bias a coverage-denial attack without ever needing the key. Security reduces entirely to secrecy of K_s and integrity of ID; an adversary holding K_s can re-watermark manipulated content so it verifies perfectly. The authentication tag (6) covers a block's complete 128-bit stored payload – its own content and the recovery descriptor bits it carries – so tampering with either half is detected at that block; what it does not cover is whether a flagged block can be reconstructed, which depends on its mapped partner also being intact and is a property of the mapping's coverage, not of authentication. The scheme authenticates against modification by parties without the key, and nothing more.

VII. Conclusion and Future Scope

This paper presented a complete, deterministic, training-free framework for watermark-guided tamper localization and recovery based on self-embedding fragile watermarking. The design partitions the image into 8×8 blocks and allocates the 128 bits available in two LSB planes between a 32-bit keyed authentication tag stored in-block and a 96-bit compressed recovery descriptor stored in a spatially distant partner block. Computing both payloads from the six MSB planes makes the construction self-consistent without iteration; keying the tag to block index and image identifier defends against block-wise collage counterfeiting [5]; generating the mapping as a key-seeded single-cycle permutation with enforced minimum separation implements the distant-mapping principle for avoiding the tamper coincidence problem [20], guaranteeing full recoverability for attacks more compact than $d_{\min}$.

The methodological contribution is the joint evaluation protocol: over four realistic tamper classes at three tamper ratios, with exact self-generated ground truth, the framework reports imperceptibility, localization precision/recall/F1/IoU, recoverability rate, and in-region recovery fidelity in a single view, making visible a trade-off the fragile-watermarking literature has often reported one half at a time. Measured over the full grid (32 images, both descriptor variants, four tamper classes at three tamper ratios, a null condition, and a block-size ablation – 1,184 rows in total, 23 of 23 automated sanity checks passing with zero warnings), the scheme achieves block-level recall exactly 1.0000 on the 768 main-grid rows, up from 0.9999 before the descriptor-binding fix eliminated the MSB-preserved miss category entirely (block-level precision is a

tautology of the block-level ground-truth rule and is not reported as a result; pixel-level precision, which does vary, averages 0.9489), zero false positives across 1.8 million null-condition block verifications – a determinism and implementation-consistency check yielding a rule-of-three 95% upper bound of 1.66×10^{-6} on the per-block false-positive rate, not a bound on the 2^{-32} cryptographic false-accept probability – and in-region recovery fidelity that stays essentially flat at 28.4–29.0 dB across tamper ratios even as whole-image quality falls from 33.79 dB to 17.39 dB with shrinking coverage ρ – confirming coverage, not descriptor fidelity, as the governing factor on whole-image recovery, and confirming that closing the gap to reference-sharing and fountain-coded schemes' graceful degradation (Future Scope item 1 below) is the highest-value next step for this design.

Target applications include digital forensic evidence archives, journalism/media authentication at ingest, medical imaging integrity in PACS, legal and official document scans, insurance claim verification, and self-authenticating camera/device firmware.

Extensions follow directly. Reference sharing: replacing one-to-one backup with a shared-reference or fountain-coded allocation [8], [9] would raise the tolerable tamper ratio beyond the structural $1 - \alpha$ coverage limit – the highest-value single upgrade. Hierarchical multi-scale verification [4] would decouple localization resolution from per-block payload capacity. Semi-fragile extension embedding authentication in quantization-invariant DCT coefficient relationships [3], while retaining LSB-domain recovery data, would allow tolerance of a bounded, declared degree of JPEG re-compression – the single change that would most widen the deployment envelope. Reversible embedding would allow bit-exact restoration of verified-authentic images. Second-stage harmonic fill for blocks still unrecoverable after (11) (interpolating from surviving neighbours rather than leaving them explicitly marked) is a plausible extension; at present it exists only as an unimplemented design note and is not evaluated in this paper, so it must be kept clearly separate from – and never merged into – the primary recovery figures reported here. Rate-adaptive descriptors could allocate more bits to perceptually or semantically salient blocks (e.g. a small unsupervised vector-quantization codebook over block patches in place of fixed DCT truncation, trained with no labels and no GPU) under a global budget, turning descriptor design into an explicit rate–distortion optimization. Extension to colour and video would provide a larger, more attack-resilient space for recovery data via inter-frame mapping. Finally, hybrid operation with passive forensics – using the watermark's exact mask, where intact, as supervision or a trusted prior for a learned detector operating on the same content – offers a path combining the assurance of active authentication with the coverage of blind analysis.

References

[1] J. Fridrich and M. Goljan, "Protection of digital images using self-embedding," in Proc. Symp. Content Security and Data Hiding in Digital Media, Newark, NJ, USA, May 1999.

[2] J. Fridrich and M. Goljan, "Images with self-correcting capabilities," in Proc. IEEE Int. Conf. Image Process. (ICIP), vol. 3, Kobe, Japan, Oct. 1999, pp. 792–796.

[3] C.-Y. Lin and S.-F. Chang, "Semi-fragile watermarking for authenticating JPEG visual content," in Proc. SPIE 3971, Security and Watermarking of Multimedia Contents II, 2000, pp. 140–151. DOI: 10.1117/12.384968

[4] P.-L. Lin, C.-K. Hsieh, and P.-W. Huang, "A hierarchical digital watermarking method for image tamper detection and recovery," Pattern Recognit., vol. 38, no. 12, pp. 2519–2529, Dec. 2005.

[5] M. Holliman and N. Memon, "Counterfeiting attacks on oblivious block-wise independent invisible watermarking schemes," IEEE Trans. Image Process., vol. 9, no. 3, pp. 432–441, Mar. 2000. DOI: 10.1109/83.826780

[6] J. Fridrich, "Security of fragile authentication watermarks with localization," in Proc. SPIE 4675, Security and Watermarking of Multimedia Contents IV, San Jose, CA, USA, Jan. 2002, pp. 691–700. DOI: 10.1117/12.465330

[7] X. Zhang and S. Wang, "Fragile watermarking with error-free restoration capability," IEEE Trans. Multimedia, vol. 10, no. 8, pp. 1490–1499, Dec. 2008.

[8] X. Zhang, S. Wang, Z. Qian, and G. Feng, "Reference sharing mechanism for watermark self-embedding," IEEE Trans. Image Process., vol. 20, no. 2, pp. 485–495, Feb. 2011.

[9] P. Korus and A. Dziech, "Efficient method for content reconstruction with self-embedding," IEEE Trans. Image Process., vol. 22, no. 3, pp. 1134–1147, Mar. 2013. DOI: 10.1109/TIP.2012.2227769

[10] P. W. Wong and N. Memon, "Secret and public key image watermarking schemes for image authentication and ownership verification," IEEE Trans. Image Process., vol. 10, no. 10, pp. 1593–1601, Oct. 2001.

[11] C. Rey and J.-L. Dugelay, "A survey of watermarking algorithms for image authentication," EURASIP J. Appl. Signal Process., vol. 2002, no. 6, pp. 613–621, 2002.

[12] P. Zhou, X. Han, V. I. Morariu, and L. S. Davis, "Learning rich features for image manipulation detection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, Jun. 2018.

[13] Y. Wu, W. AbdAlmageed, and P. Natarajan, "ManTra-Net: Manipulation tracing network for detection and localization of image forgeries with anomalous features," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, Jun. 2019.

[14] I. J. Cox, J. Kilian, F. T. Leighton, and T. Shamoon, "Secure spread spectrum watermarking for multimedia," IEEE Trans. Image Process., vol. 6, no. 12, pp. 1673–1687, Dec. 1997.

[15] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, Apr. 2004.

[16] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS Publication 180-4, Aug. 2015.

[17] University of Southern California, Signal and Image Processing Institute, "The USC-SIPI Image Database." [Online]. Available: https://sipi.usc.edu/database/

[18] A. Aminuddin and F. Ernawan, "AuSR1: Authentication and self-recovery using a new image inpainting technique with LSB shifting in fragile image watermarking," J. King Saud Univ. – Comput. Inf. Sci., 2022. DOI: 10.1016/j.jksuci.2022.02.004

[19] A. Aminuddin and F. Ernawan, "AuSR2: Image watermarking technique for authentication and self-recovery with image texture preservation," Comput. Electr. Eng., vol. 102, art. 108207, 2022. DOI: 10.1016/j.compeleceng.2022.108207

[20] A. Aminuddin and F. Ernawan, "AuSR3: A new block mapping technique for image authentication and self-recovery to avoid the tamper coincidence problem," J. King Saud Univ. – Comput. Inf. Sci., vol. 35, no. 9, art. 101755, 2023.

[21] A. Aminuddin, F. Ernawan, D. Nincarean, A. Amrullah, and D. Ariatmanto, "TCBR and TCBD: Evaluation metrics for tamper coincidence problem in fragile image watermarking," Eng. Sci. Technol. – Int. J., vol. 56, art. 101790, 2024.
[22] X. Zhang, S. Wang, and G. Feng, "Fragile watermarking scheme with extensive content restoration capability," in Digital Watermarking (IWDW 2009), Lecture Notes in Computer Science, vol. 5703. Berlin, Germany: Springer, 2009.
[23] "A recent survey of self-embedding fragile watermarking scheme for image authentication with recovery capability," EURASIP J. Image Video Process., 2019. DOI: 10.1186/s13640-019-0462-3
[24] "Security analysis of a self-embedding fragile image watermark scheme," arXiv:1812.11735, 2018.
[25] X. Zhang, R. Li, J. Yu, Y. Xu, W. Li, and J. Zhang, "EditGuard: Versatile image watermarking for tamper localization and copyright protection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2024. arXiv:2312.08883.
[26] "DeepMark: A proactive deep learning-based watermarking model for tamper detection and localization across images and videos," Inf. Process. Manage., 2026.
[27] "RecoverMark: Robust watermarking for localization and recovery of manipulated faces," 2026.